

Quiz — 2.3.1 Algorithms

Name: _____ Date: _ **Mark:** ____ / 35

OCR H446 · Component 02 · 2.3.1 — show your traces

Section A — Multiple choice (10 marks)

A1. What is the worst-case time complexity of **binary search**? A. $O(1)$ B. $O(\log n)$ C. $O(n)$ D. $O(n \log n)$

Your answer: ☐

A2. What is the **average** time complexity of **merge sort**? A. $O(n)$ B. $O(n \log n)$ C. $O(n^2)$ D. $O(2^n)$

Your answer: ☐

A3. What is the **worst-case** time complexity of **quick sort**? A. $O(\log n)$ B. $O(n \log n)$ C. $O(n^2)$ D. $O(2^n)$

Your answer: ☐

A4. A list must be in a particular state before **binary search** can be used. What is it? A. The list must be sorted B. The list must contain only integers C. The list must have an even number of items D. The list must be stored in a linked list

Your answer: ☐

A5. Which data structure does a **breadth-first traversal** use to keep track of nodes still to visit? A. Stack

B. Queue C. Binary tree D. Hash table

Your answer: ☐

A6. Simplify the expression $4n^2 + 200n + 9$ to Big O notation. A. $O(n)$ B. $O(200n)$ C. $O(n^2)$ D. $O(4n^2)$

Your answer: ☐

A7. What is the time complexity of **pushing** an item onto a **stack**? A. $O(1)$ B. $O(\log n)$ C. $O(n)$ D. $O(n^2)$

Your answer: ☐

A8. Which traversal of a **binary search tree** outputs the values in **ascending order**? A. Pre-order B. In-order

C. Post-order D. Breadth-first

Your answer: ☐

A9. A problem that can only be solved in **exponential time or worse** is described as: A. Tractable B.

Intractable C. Logarithmic D. Stable

Your answer: ☐

A10. What is the key difference between **A*** and **Dijkstra's** algorithm? A. A can handle negative edge weights B. A uses a heuristic estimate of the remaining cost to the goal C. A explores every node in the graph; Dijkstra does not D. A does not guarantee finding a path

Your answer: ☐

Section B — Traces & short answer (20 marks)

B1. A program contains the following nested loop structure over a list of n items:

```
for i in range(n):  
    for j in range(n):  
        compare items
```

State the Big O time complexity and **justify** your answer. [3]

.....

.....

.....

B2. Trace a **bubble sort** (one pass = comparing adjacent pairs left-to-right, swapping if out of order) on the list below. Show the list **after each complete pass**. [4]

Start: [5, 3, 8, 1, 4]

After pass	Item 1	Item 2	Item 3	Item 4	Item 5
Pass 1					
Pass 2					
Pass 3					
Pass 4					

B3. A **binary search** is performed on the sorted list below to find the value **23**.

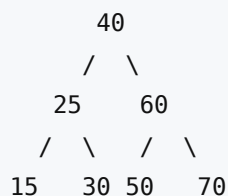
[4, 9, 15, 18, 23, 27, 31, 42, 56] (indices 0–8)

Complete the trace table. Show each step until the value is found. [4]

Step	low	high	mid (index)	value at mid	< = > target?
1					
2					
3					

State the result:

B4. Consider the binary tree below.



Write the output of each traversal. [3]

(a) **Pre-order:** [1]

(b) **In-order:** [1]

(c) **Post-order:** [1]

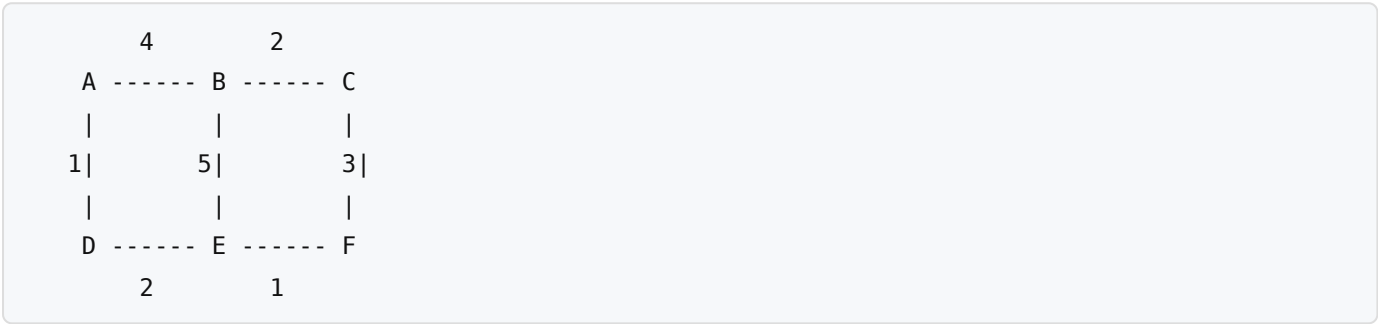
B5. State which traversal type (DFS or BFS) uses a **stack** and which uses a **queue**, and give **one** practical use of a breadth-first traversal. **[3]**

.....

.....

.....

B6. Dijkstra's algorithm is run on the weighted graph below, starting at node **A**, to find the shortest distance to every other node.



Edges (undirected, with weights): A–B = 4, A–D = 1, B–C = 2, B–E = 5, C–F = 3, D–E = 2, E–F = 1

Complete the table of **final shortest distances** from A. **[3]**

Node	Shortest distance from A	Via (previous node)
A	0	—
B		
C		
D		
E		
F		

Section C — Exam-style question (5 marks)

C1. A games developer needs an algorithm to find the shortest route for a non-player character (NPC) to travel from its current position to a **single, known target location** on a large map.

Compare the suitability of **Dijkstra's algorithm** and the **A*** algorithm for this task, and justify which you would recommend. **[5]**

.....

.....

.....

.....

Answer key

Section A (10 marks — 1 each)

Q	Ans	Why
A1	B	Binary search halves the search space each step → $O(\log n)$ worst case.
A2	B	Merge sort divides into $\log n$ levels, each doing n work to merge → $O(n \log n)$ in all cases.
A3	C	Quick sort degrades to $O(n^2)$ with a poor pivot (e.g. already-sorted data); average is $O(n \log n)$.
A4	A	Binary search requires a sorted list so a half can be safely discarded each pass.
A5	B	BFS uses a queue (FIFO) to process nodes level by level. (DFS uses a stack.)
A6	C	Drop constants and lower-order terms; the dominant term n^2 gives $O(n^2)$.
A7	A	Stack push/pop/peek are $O(1)$ — they act only on the top, no traversal.
A8	B	In-order (Left → Node → Right) of a BST yields values in ascending order.
A9	B	Only solvable in exponential time or worse = intractable .
A10	B	A* adds an admissible heuristic $h(n)$ estimating remaining cost to the goal ($f = g + h$), directing the search.

Section B (20 marks)

B1 [3] — $O(n^2)$. (1 mark) Justification (2 marks): There is a loop nested inside a loop. The outer loop runs n times, and for each outer iteration the inner loop runs n times, giving roughly $n \times n = n^2$ comparisons. As constants and lower-order terms are dropped, this is **$O(n^2)$** (quadratic) — doubling n quadruples the work.

B2 [4] — Bubble sort passes (largest value bubbles to the right each pass). Award 1 mark per correct pass row.

Start: [5, 3, 8, 1, 4]

After pass	Result
Pass 1	[3, 5, 1, 4, 8]
Pass 2	[3, 1, 4, 5, 8]
Pass 3	[1, 3, 4, 5, 8]
Pass 4	[1, 3, 4, 5, 8] (no swaps — sorted)

Working for Pass 1: compare 5,3 → swap [3,5,8,1,4] ; 5,8 → no swap; 8,1 → swap [3,5,1,8,4] ; 8,4 → swap [3,5,1,4,8] . Pass 2: 3,5 → no; 5,1 → swap [3,1,5,4,8] ; 5,4 → swap [3,1,4,5,8] ; 5,8 → no.

Pass 3: 3,1 → swap [1, 3, 4, 5, 8] ; 3,4 → no; 4,5 → no; 5,8 → no. Pass 4: all in order, no swaps.

B3 [4] — Binary search for **23** in [4, 9, 15, 18, 23, 27, 31, 42, 56] . Award up to 4 for a correct trace.

Step	low	high	mid (index)	value at mid	vs target 23
1	0	8	4	23	= → found

Note: mid = (0 + 8) DIV 2 = 4; value at index 4 is 23, which equals the target, so the search succeeds on the **first** step. Result: **23 found at index 4** (1 mark). (Full marks for showing low=0, high=8, mid=4, identifying value 23, and concluding "found at index 4". If a candidate's list/target leads to multiple steps, the same method — recompute mid, discard a half by adjusting low/high — earns the marks.)

B4 [3] — Tree: root 40; left subtree 25 (children 15, 30); right subtree 60 (children 50, 70).

(a) **Pre-order** (Node → Left → Right): **40, 25, 15, 30, 60, 50, 70** (1) (b) **In-order** (Left → Node → Right): **15, 25, 30, 40, 50, 60, 70** (1) (c) **Post-order** (Left → Right → Node): **15, 30, 25, 50, 70, 60, 40** (1)

B5 [3] — - **Depth-first traversal (DFS)** uses a **stack** (or recursion, which uses the call stack). (1) - **Breadth-first traversal (BFS)** uses a **queue**. (1) - One use of BFS: finding the **shortest path in an unweighted graph** (also accept: web crawling, social-network "degrees of separation", peer-to-peer searching). (1)

B6 [3] — Dijkstra from A. Final shortest distances:

Node	Shortest distance from A	Via
A	0	—
D	1	A
E	3	D
F	4	E
B	4	A
C	6	B

Working: - A = 0 (start). - A–D = 1, A–B = 4. Settle **D = 1** (smallest). - From D: E = 1 + 2 = **3**. Settle **E = 3**. - From E: F = 3 + 1 = **4**; B via E = 3 + 5 = 8 (worse than 4, keep B = 4). Settle **B = 4** and **F = 4** (both 4; either order). - From B: C = 4 + 2 = **6**. From F: C via F = 4 + 3 = 7 (worse, keep 6). Settle **C = 6**.

Award up to 3 marks: B=4, C=6, D=1, E=3, F=4 (e.g. 1 mark per 2 correct nodes, rounded in candidate's favour). Common correct alternative "via" for C is B; for F is E.

Section C (5 marks)

C1 [5] — Indicative content; award marks for valid comparison points and a justified recommendation. Mark band 4–5 for a developed comparison that explicitly justifies the choice using the single-target nature of the task.

- Both algorithms are **guaranteed to find the optimal (shortest) path** when edge weights are non-negative (*A requires an admissible heuristic that never overestimates*). (1)*

- **Dijkstra** uses only the actual cost so far, $g(n)$, and searches **outwards in all directions**, finding the shortest path to **all** nodes. This explores **more nodes than necessary** when only one target is needed. (1)
 - **A*** uses $f(n) = g(n) + h(n)$, adding a heuristic estimate $h(n)$ of the remaining distance to the goal (e.g. straight-line/Manhattan distance). This **directs the search towards the target**. (1)
 - Because the target is a **single, known location**, *A typically explores far fewer nodes and so is faster / more efficient, which matters on a large map in a real-time game where performance is critical*. (1)*
 - **Recommendation: A*** is the better choice here, as it gives the same optimal route as Dijkstra but reaches a single known goal more efficiently. (Dijkstra would be preferable if shortest paths to *many* destinations were required, or if no good heuristic were available.) (1)
-

Total: 35 marks (Section A = 10, Section B = 3 + 4 + 4 + 3 + 3 + 3 = 20, Section C = 5).